



Smart Residential Mess Monitoring and Food Management System

Course : Java Programming
Fall Semester 26-27

Submitted by

Sahil Kumar
25BAI11048
Branch: CSE (AI&ML)

Introduction

Residential hostels and university campuses face recurring challenges in dining facility (mess) administration. Unpredictable student turnout leads to severe food waste, manual paper-based attendance causes billing discrepancies, and kitchen staff lack data to plan quantities accurately.

The **Smart Residential Mess Monitor** bridges hardware and web software to automate meal tracking , log live food waste using load-cell sensors, provide a digital portal for students to opt in/out of meals, and give administrators predictive consumption insights.

Problem Statement

- ❖ **Massive Food Waste:** Inflexible meal prep quotas cause 20–35% excess food disposal daily.
- ❖ **Manual Record Errors:** Physical register logbooks lead to meal slip manipulation, duplicate billing, and sluggish queue throughput during peak hours.
- ❖ **Lack of Real-Time Feedback:** Wardens and caterers receive no real-time telemetry on leftover inventory, menu popularity, or mess hygiene/stock conditions.

Functional Requirements

- ❑ **User Authentication & Profiles:** Role-based access for Students, Mess Managers, Kitchen Staff, and Admins.
- ❑ **Meal Opt-In/Opt-Out Management:** Students can mark leaves or cancel upcoming meals up to 3 hours prior to cooking cycles.
- ❑ **Digital Attendance & Gate Check-in:** Real-time QR/Rfid authentication at mess entry points to validate active meal plans and decrement allowances.
- ❑ **Automated Waste Monitoring:** Smart bin integration equipped with load-cell sensors to automatically capture and record post-meal discarded food weight.
- ❑ **Daily Menu & Rating System:** Dynamic menu updates and instantaneous student ratings/feedback per meal.
- ❑ **Analytics & Reports:** Automated calculation of daily consumption ratios, leftover metrics, and student monthly mess bills.

Non-Functional Requirements

Performance & Latency: Check-in authentication must execute in < 1.5 seconds per student to prevent entry bottlenecks.

Reliability & Availability: System uptime $\geq 99.5\%$ during scheduled dining windows (Breakfast, Lunch, Dinner).

Security: Secure password hashing , Session handling, and role-based access control (RBAC).

Scalability: Capable of handling concurrent check-ins for 1,000+ active residents simultaneously.

Usability: Mobile-first responsive UI for students; high-contrast dashboard for mess counters.

System Architecture

The system adopts a modern **Three-Tier Architecture**:

- **Presentation Tier:** Responsive Web & Mobile frontend built with React.js or Flutter.
- **Application Tier:** RESTful API backend using Node.js/Express or Python/FastAPI managing business logic, notification triggers, and edge communication.
- **Data Tier:** Relational database (PostgreSQL/MySQL) storing transaction records, student accounts, and sensor logs.
- **Edge / IoT Layer:** ESP32/Arduino microcontroller connected to an RC522 RFID reader and an HX711 Load Cell amplifier for waste bins, publishing logs via MQTT/HTTPS.

Design Diagrams

Use Case Diagram:

- *Actors:* Student, Mess Manager, Chef/Kitchen Staff, System Admin.
- *Use Cases:* Mark Meal Attendance, Skip Meal, Log Waste Weight, Publish Menu, Review Analytics, View Monthly Bill.

Workflow Diagram:

- Student arrives at mess ,Scan's RFID tag ,System verifies opt-in status, Green signal/Access granted ,Meal consumed , Discarded waste placed on Smart Bin scale, Weight logged to cloud server.

Sequence Diagram (Meal Validation):

1. **Student** taps card on **RFID Reader**.
2. **Database** returns verification status.

Component Diagram:

- **Client App** , **API Gateway** , [**Auth Service**, **Meal Manager**, **IoT Ingestion Service**, **Reporting Engine**] ,**Database**.

ER Diagram:

- **Users:** **user_id** , **name**, **email**,
- **Meals:** **meal_id** , **date**, **meal_type** (Breakfast/Lunch/Dinner), **menu_description**.
- **MealBookings:** **booking_id** , **user_id** , **meal_id** , **status** (Opted-in/Opted-out/Attended), **timestamp**.
- **WasteLogs:** **log_id** , **meal_id** , **waste_weight_kg**, **recorded_at**

Design Decisions & Rationale

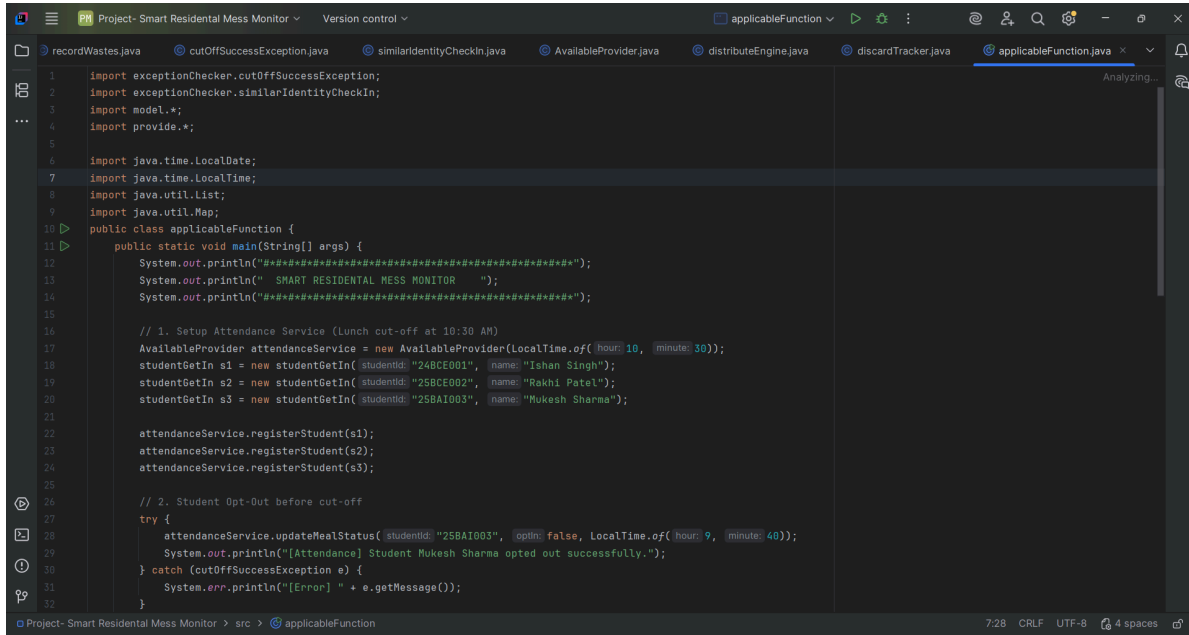
- **Relational DB (PostgreSQL) over NoSQL:** Financial mess billing, meal audits, and attendance demand strict ACID compliance and relational integrity.
- **RFID over Face Recognition:** RFID cards are faster, low-cost, unaffected by ambient mess lighting, and respect student biometric privacy.
- **REST APIs with JWT:** Stateless authentication reduces server memory overhead across hundreds of concurrent check-ins.

Implementation Details

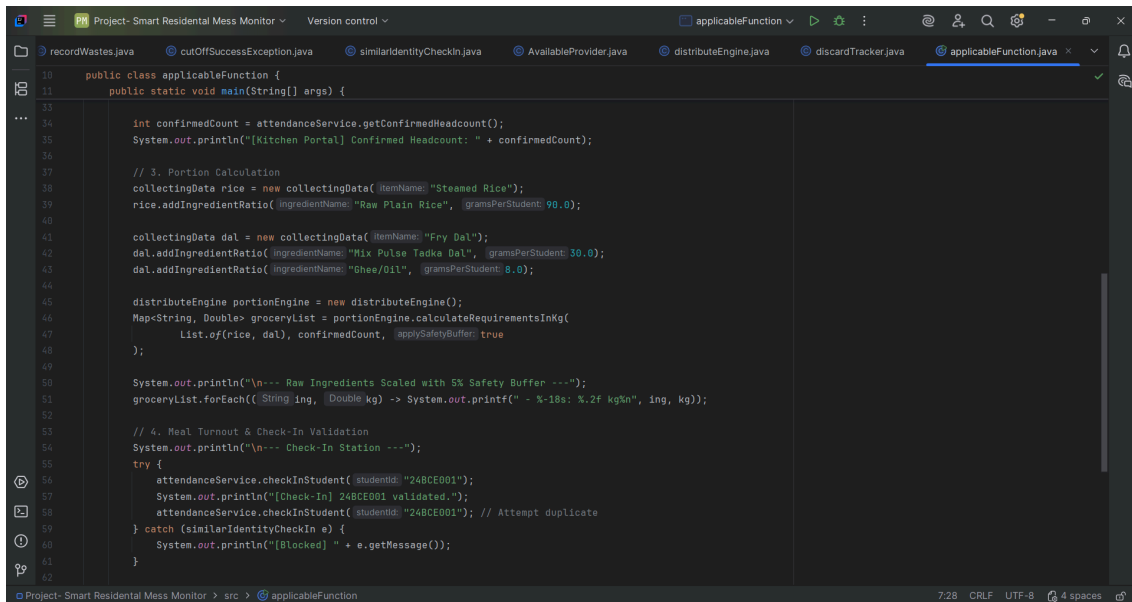
The system is implemented as a standalone Java application using standard object-oriented principles. It manages student meal authentication, opt-outs, and waste logging through an in-memory data store.

- **Language:** Java Standard Edition (Core Java / JDK)
- **Design Pattern:** Modular class-based design (Entities + Manager + Console Interface)
- **Data Storage:** In-memory collections (`ArrayList`)
- **Input Interface:** `java.util.Scanner` for terminal/console input simulation

Screenshots/ Results



```
1 import exceptionChecker.cutOffSuccessException;
2 import exceptionChecker.similarIdentityCheckIn;
3 import model.*;
4 import provide.*;
5
6 import java.time.LocalDate;
7 import java.time.LocalDateTime;
8 import java.util.List;
9 import java.util.Map;
10
11 public class applicableFunction {
12     public static void main(String[] args) {
13         System.out.println("#####");
14         System.out.println(" SMART RESIDENTAL MESS MONITOR ");
15         System.out.println("#####");
16
17         // 1. Setup Attendance Service (Lunch cut-off at 10:30 AM)
18         AvailableProvider attendanceService = new AvailableProvider(LocalTime.of(10, 30));
19         studentGetIn s1 = new studentGetIn( studentId: "24BCE001", name: "Ishan Singh");
20         studentGetIn s2 = new studentGetIn( studentId: "25BCE002", name: "Rakhi Patel");
21         studentGetIn s3 = new studentGetIn( studentId: "25BA1003", name: "Mukesh Sharma");
22
23         attendanceService.registerStudent(s1);
24         attendanceService.registerStudent(s2);
25         attendanceService.registerStudent(s3);
26
27         // 2. Student Opt-Out before cut-off
28         try {
29             attendanceService.updateMealStatus( studentId: "25BA1003", optIn: false, LocalDateTime.of(10, 40));
30             System.out.println("[Attendance] Student Mukesh Sharma opted out successfully.");
31         } catch (cutOffSuccessException e) {
32             System.err.println("[Error] " + e.getMessage());
33         }
34     }
35 }
```



```
36
37     int confirmedCount = attendanceService.getConfirmedHeadcount();
38     System.out.println("[Kitchen Portal] Confirmed Headcount: " + confirmedCount);
39
40     // 3. Portion Calculation
41     collectingData rice = new collectingData( itemName: "Steamed Rice");
42     rice.addIngredientRatio( ingredientName: "Raw Plain Rice", gramsPerStudent: 90.0);
43
44     collectingData dal = new collectingData( itemName: "Fry Dal");
45     dal.addIngredientRatio( ingredientName: "Mix Pulse Tadka Dal", gramsPerStudent: 30.0);
46     dal.addIngredientRatio( ingredientName: "Ghee/Oil", gramsPerStudent: 8.0);
47
48     distributeEngine portionEngine = new distributeEngine();
49     Map<String, Double> groceryList = portionEngine.calculateRequirementsInKg(
50         List.of(rice, dal), confirmedCount, applySafetyBuffer: true
51     );
52
53     System.out.println("\n--- Raw Ingredients Scaled with 5% Safety Buffer ---");
54     groceryList.forEach((String ing, Double kg) -> System.out.printf(" ~ 18s: %.2f kg\n", ing, kg));
55
56     // 4. Meal Turnout & Check-In Validation
57     System.out.println("\n--- Check-In Station ---");
58     try {
59         attendanceService.checkInStudent( studentId: "24BCE001");
60         System.out.println("[Check-In] 24BCE001 validated.");
61         attendanceService.checkInStudent( studentId: "24BCE001"); // Attempt duplicate
62     } catch (similarIdentityCheckIn e) {
63         System.out.println("[Blocked] " + e.getMessage());
64     }
65 }
```

```
Project- Smart Residential Mess Monitor  Version control  applicableFunction  applicableFunction.java x
10 public class applicableFunction {
11     public static void main(String[] args) {
51         groceryList.forEach((String ing, Double kg) -> System.out.printf(" - %-18s: %.2f kg\n", ing, kg));
52
53         // 4. Meal Turnout & Check-In Validation
54         System.out.println("\n--- Check-In Station ---");
55         try {
56             attendanceService.checkInStudent(studentId: "248CE001");
57             System.out.println("[Check-In] 248CE001 validated.");
58             attendanceService.checkInStudent(studentId: "248CE001"); // Attempt duplicate
59         } catch (similarIdentityCheckIn e) {
60             System.out.println("[Blocked] " + e.getMessage());
61         }
62
63         // 5. Post-Meal Waste Logging
64         discardTracker wasteService = new discardTracker();
65         wasteService.logWaste(new recordWastes(LocalDate.now(), mealType: "Lunch", wastedWeightKg: 3.2, estimatedCostLost: 480.0));
66
67         System.out.println("\n--- Daily Waste Audit ---");
68         wasteService.getAllLogs().forEach(System.out::println);
69         System.out.printf("Total Waste: %.2f kg | Total Loss: ₹%.2f\n",
70             wasteService.getTotalWastedWeight(), wasteService.getTotalFinancialLoss());
71     }
72 }
73
74
75
Project- Smart Residential Mess Monitor > src > applicableFunction 7:28 CRLF UTF-8 4 spaces
```

```
Project- Smart Residential Mess Monitor  Version control  applicableFunction  applicableFunction.java x
Run  applicableFunction x
...
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.6.2\lib\idea_rt.jar=60372" -Dfile.encoding=UTF-8
#####
SMART RESIDENTIAL MESS MONITOR
#####
[Attendance] Student Mukesh Sharma opted out successfully.
[Kitchen Portal] Confirmed Headcount: 2

--- Raw Ingredients Scaled with 5% Safety Buffer ---
- Raw Plain Rice : 0.19 kg
- Ghee/Oil : 0.02 kg
- Mix Pulse Tadka Dal: 0.06 kg

--- Check-In Station ---
[Check-In] 248CE001 validated.
[Blocked] Student 248CE001 has already checked in!

--- Daily Waste Audit ---
[2026-09-12 - Lunch] Waste: 3.20 kg | Loss: ₹480.00
Total Waste: 3.20 kg | Total Loss: ₹480.00

Process finished with exit code 0

Project- Smart Residential Mess Monitor > src > applicableFunction 7:28 CRLF UTF-8 4 spaces
```

Testing Approach

The testing strategy uses automated unit tests written in pure Java. A dedicated test driver class executes test cases against the core methods of the mess system, checking outputs against expected conditions and printing clear PASS/FAIL results to the console.

Test Strategy & Cases

- **TC-01 (Valid Entry):** An active student with sufficient tokens is granted entry, and their token count decreases by 1.
- **TC-02 (Opt-Out Block):** A student who opted out of a meal is blocked from entering.
- **TC-03 (Zero Balance Block):** A student with 0 tokens is denied entry.
- **TC-04 (Invalid Card):** An unregistered RFID tag is flagged and rejected.
- **TC-05 (Waste Logging):** Kitchen staff can successfully record food waste weight.

Challenging Faced

Predictive Analytics & Computer Vision Complexity

- **Food Waste Monitoring:** If tracking mess cleanliness or uneaten food via image recognition, Java lacks the rich, native ecosystem that Python possesses (OpenCV/PyTorch bindings exist via JavaCV, but they are less streamlined and harder to deploy).
- **Demand Forecasting:** Predicting headcount based on menu items requires integrating machine learning models, which usually requires running lightweight ONNX runtimes inside Java or communicating with an external microservice.

